

Name: _____

Class: _____

Mr. Rawson • GCSE Computer Science



Year 11 2026

Programming 6 — Arrays — **ANSWERS**

Why we need them

You want to store the surnames of four people. With what you know so far, that takes four variables.

```
name1 = "Long"
name2 = "Boden"
name3 = "Abraham"
name4 = "Schmidt"
```

That works for four. For a class of thirty it is thirty lines, and to print them all you would write thirty `print()` statements. An **array** keeps them all together under **one** name.

VOCAB Array (OCR 2.2.3) — one variable that holds many values, all of the same data type, each in a numbered position

1 Make an array and read one box

The rule. Put the values inside square brackets, separated by commas. Each value gets a number called its **index**. The index starts at **0**.

```
surnames = ["Long", "Boden", "Abraham", "Schmidt"]
```

Index	0	1	2	3
Value	"Long"	"Boden"	"Abraham"	"Schmidt"

This is the same row of numbered boxes as a string in Programming 5. A string holds one character in each box. An array holds a whole value in each box.

VOCAB Index (OCR 2.2.3) — the position number of a box in an array. The first box is index 0

To read a box, write the array's name and the index in square brackets.

You type	Prints	What it does
<code>print(surnames[0])</code>	Long	Reads box 0, the first one.
<code>print(surnames[3])</code>	Schmidt	Reads box 3, the last one.
<code>print(len(surnames))</code>	4	Counts the boxes, exactly as it did for a string.

The last index is always one less than the length. Four boxes, numbered 0, 1, 2, 3.

Now try • Make an array called `primes` holding the first five prime numbers: 2, 3, 5, 7, 11. Numbers go in without speech marks. Then print the first prime, the last prime, and the length of the array.

Check • Three lines come out: 2, then 11, then 5.

ANSWER

```
primes = [2, 3, 5, 7, 11]
print(primes[0])          # 2
print(primes[4])          # 11 (five boxes, so the last index is 4)
print(len(primes))        # 5
```

2 Change one box

The rule. Put the box on the left of `=` and the new value on the right. The other boxes stay exactly as they were.

```
surnames = ["Long", "Boden", "Abraham", "Schmidt"]
surnames[1] = "Wheeler"
print(surnames)
```

Line 2 reads as “box 1 becomes `"Wheeler"`”. Line 3 prints the whole array at once: `['Long', 'Wheeler', 'Abraham', 'Schmidt']`. Only box 1 has changed.

Now try • Start from your `primes` array. Change box 2 to 13, then print the whole array.

Check • `[2, 3, 13, 7, 11]`. The 5 has gone and everything else is where it was.

ANSWER

```
primes = [2, 3, 5, 7, 11]
primes[2] = 13
print(primes)          # [2, 3, 13, 7, 11]
```

3 Visit every box with a loop

The rule. A for loop counts *i* through the index numbers, and `surnames[i]` reads the box at each one. `range(len(surnames))` gives exactly the right numbers: 0, 1, 2, 3.

```
surnames = ["Long", "Boden", "Abraham", "Schmidt"]
for i in range(len(surnames)):
    print(surnames[i])
```

Part of the code

What it does

`len(surnames)`

Counts the boxes: 4.

`range(4)`

Makes the numbers 0, 1, 2, 3. It stops one short of 4, which is exactly the last index.

`for i in ...`

Runs the indented line once for each of those numbers, with *i* holding the number.

`print(surnames[i])`

Prints the box whose index is *i*.

Here is what the loop does, lap by lap.

Lap	i	surnames[i]	Prints
1st	0	surnames[0]	Long
2nd	1	surnames[1]	Boden
3rd	2	surnames[2]	Abraham
4th	3	surnames[3]	Schmidt

Add a fifth name to the array and the same two lines print all five. `len()` does the counting for you.

Now try • (a) Use a loop to print every number in `primes = [2, 3, 5, 7, 11]`, one per line.

(b) Change the print line so it prints each prime **doubled**.

Check • (a) prints 2 3 5 7 11 down the page. (b) prints 4 6 10 14 22.

ANSWER

```
primes = [2, 3, 5, 7, 11]
for i in range(len(primes)):
    print(primes[i])           # (a)

for i in range(len(primes)):
    print(primes[i] * 2)       # (b)
```

4 Fill an array from the keyboard

The rule. First make an array of the right size with every box set to 0. Then use a loop to put an input into each box.

```
scores = [0] * 5
for i in range(len(scores)):
    scores[i] = int(input("Enter a score: "))
print(scores)
```

Part of the code	What it does
scores = [0] * 5	Makes five boxes, each holding 0: [0, 0, 0, 0, 0].
for i in range(len(scores)):	Counts i through 0, 1, 2, 3, 4.
scores[i] = int(input(...))	Asks for a score, turns it into a whole number, and puts it in box i.
print(scores)	Runs once, after the loop, because it is not indented. Prints the full array.

The `int()` is the casting from Programming 3. `input()` gives back text, and `int()` makes it a number so you can do maths with it later.

Now try • Type the program above and run it. Enter 7, 9, 4, 10, 6.

Check • [7, 9, 4, 10, 6], with no speech marks, because they are numbers.

Now add a **running total** underneath, the same one you built for the self-checkout in Programming 2. Start total at 0 above the loop, add each box to it inside the loop, and print it after.

Check • With the same five scores it prints 36.

ANSWER

```
scores = [0] * 5
for i in range(len(scores)):
    scores[i] = int(input("Enter a score: "))
print(scores)                                # [7, 9, 4, 10, 6]

total = 0
for i in range(len(scores)):
    total = total + scores[i]
print(total)                                # 36
```

Stretch The user chooses how many

Write a program that:

1. asks the user **how many** test scores they want to enter
2. makes an array exactly that size
3. asks for each score and stores it
4. prints the total and the **average**.

Build it one line at a time, running after each. Line 2 is the only new idea: `[0] * n` makes `n` boxes, whatever number `n` holds.

Check • Choose 5 and enter 7, 9, 4, 10, 6. Total 36, average 7.2.

ANSWER

```
n = int(input("How many scores? "))
scores = [0] * n
for i in range(len(scores)):
    scores[i] = int(input("Enter a score: "))

total = 0
for i in range(len(scores)):
    total = total + scores[i]
print("Total: " + str(total))           # Total: 36
print("Average: " + str(total / n))     # Average: 7.2
```

Division gives a Real, so the average prints as 7.2. The `str()` is needed because `total` is a number being joined to text.

The exam writes it differently

You **type** Python. The exam paper **shows** OCR's own language, ERL. Same ideas, different words.

ERL (what you read)	Python (what you type)	Meaning
array scores[5]	scores = [0] * 5	Make 5 boxes
scores[0] = 7	scores[0] = 7	Box 0 becomes 7
scores.length	len(scores)	How many boxes
for i = 0 to scores.length - 1	for i in range(len(scores)):	Visit every box
next i	(the indent ends)	End of the loop

Read the loop row closely. ERL names the last index, so it writes `- 1`. Python's `range()` stops one short on its own. Both visit boxes 0 to 4.

One difference in the definition. In the exam, every value in an array has the same data type. A Python list is allowed to mix them, as in `["Ali", 3, True]`. Write yours the exam way, one data type per array.